

# Lecture -11

**spawn-camping attack** : In a blockchain network, these colluding miners are essentially waiting for new transactions to be generated and immediately blocking them. By doing this over and over again, they prevent legitimate activity from ever going through, which **renders the entire blockchain network completely useless**

Finality is an essential feature for ventures accepting cryptocurrencies

## POS ( System Based on Ownership )

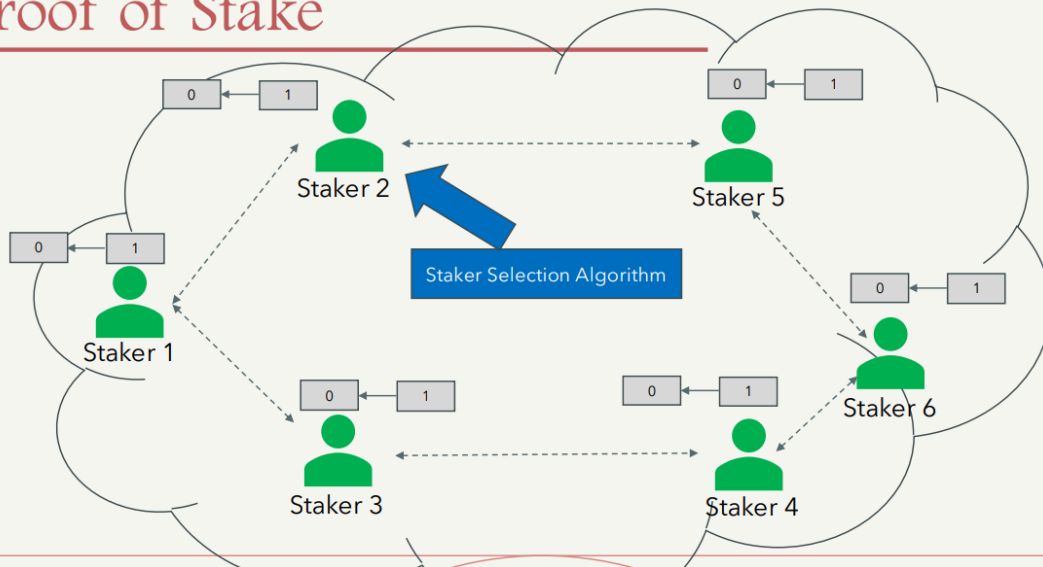
An escrow account is a legal arrangement where a neutral third party holds funds or assets on behalf of two transacting parties until contractual obligations are fulfilled

**Proof of Stake (PoS)** like putting down a financial security deposit to prove you are trustworthy. PoS relies on participants locking up their own money to secure the network.

The minter can lose the stake, in case it misbehaves  
This system has a built-in way to keep people honest.

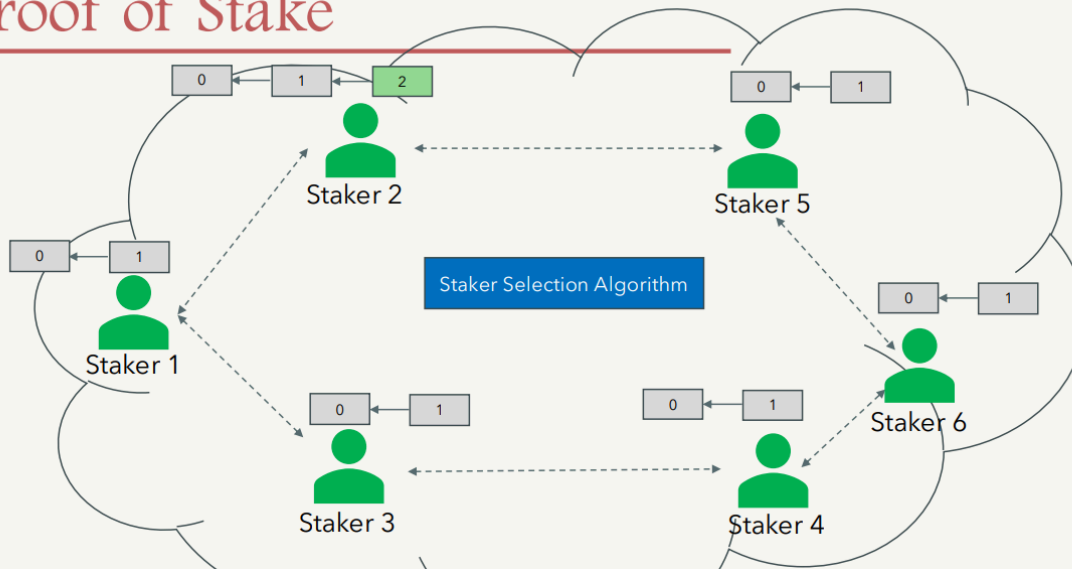
Once your money is locked up, you get a new job title. Instead of being called a "miner," the network calls you a **"Validator," "Staker,"** or **"Forger."**

## Proof of Stake



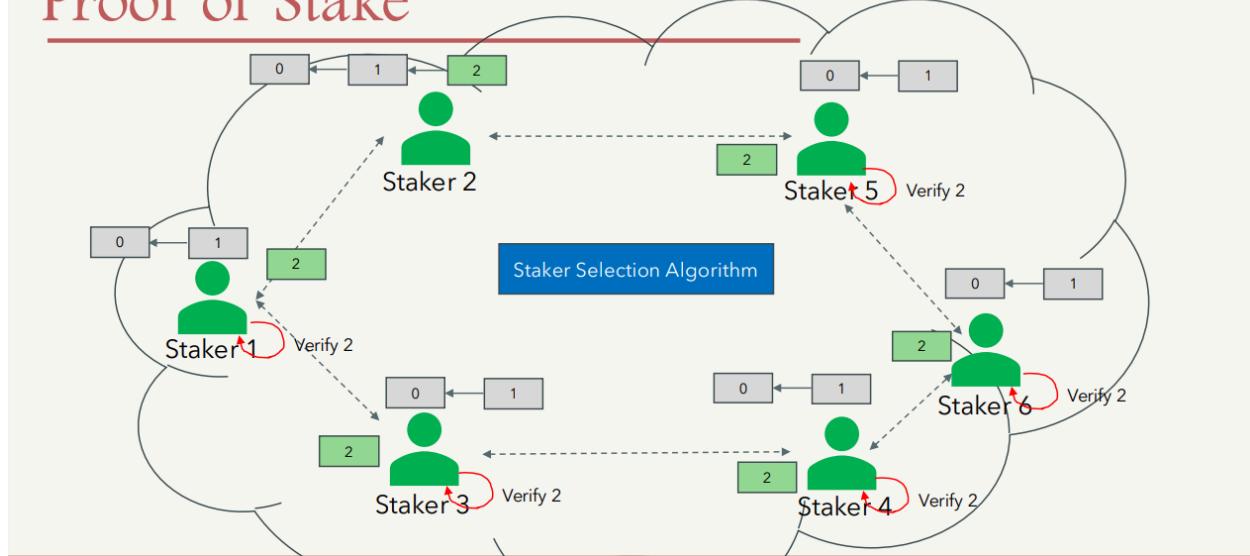
The algorithm evaluates the active pool of validators (such as Staker 1 through Staker 6) who have locked their money into the network's escrow account. Relying on the size of each validator's stake as the primary deciding factor, the algorithm chooses one specific staker (for example, Staker 2) to create the next block.

## Proof of Stake



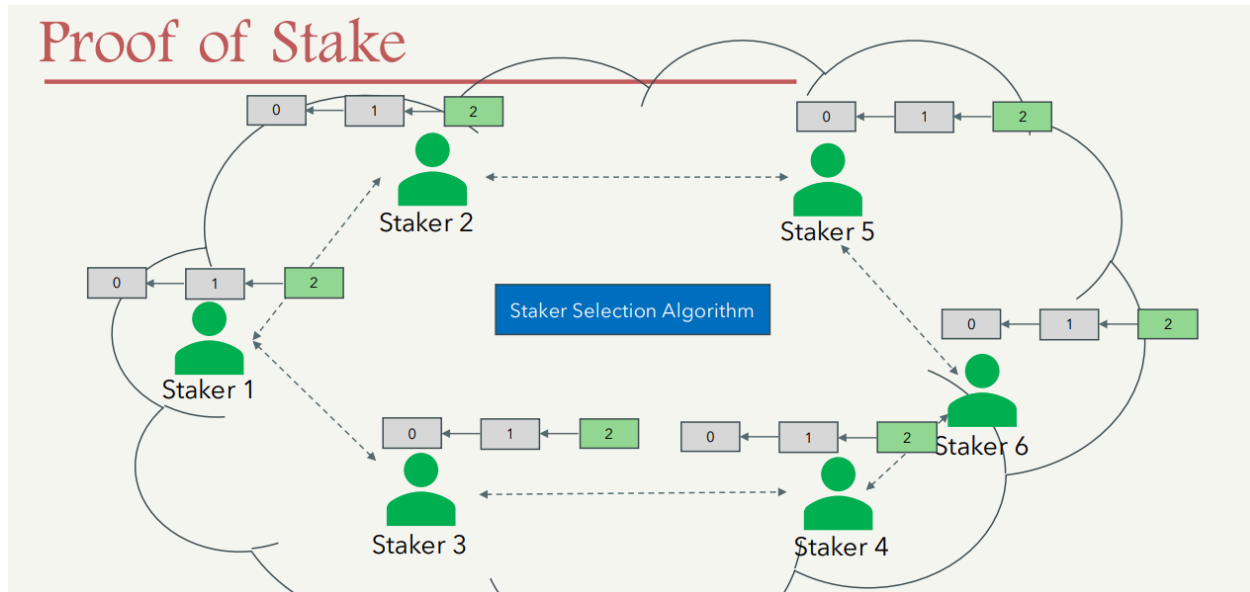
Staker 2 gathers the latest pending transactions, executes them, and packages them into a brand-new block: Block 2 (the green box). At this exact moment, only Staker 2 has this new block attached to their local chain.

## Proof of Stake



Staker 2 cannot just force the rest of the network to accept their new block. Instead, they must broadcast Block 2 across the peer-to-peer network. You will notice red circular arrows labeled "Verify 2" appearing next to Stakers 1, 3, 4, 5, and 6. They are not blindly trusting Staker 2. Each node independently runs checks to ensure Staker 2 followed the protocol.

## Proof of Stake



Because Staker 2 acted honestly and built a valid block, all the other stakers successfully complete their "Verify 2" process without finding any errors. The entire network is now back in a synchronized state. Everyone's ledger now universally reads 0 <-- 1 <-- 2.

**The "Bootstrapping" Problem** : how to fairly distribute the very first coins so that people actually have money to stake

**Pre-mining:** • A set of coins are pre-mined, which are then sold before the launch of the system in an IPO (Initial Public Offering) or ICO (Initial Coin Offering )

## DPOS

Delegates are the only entities who can propose new blocks

| 42. Difference Between PoS and DPoS |                    |                         |
|-------------------------------------|--------------------|-------------------------|
| Feature                             | PoS                | DPoS                    |
| Who validates?                      | Stakers/validators | Elected delegates       |
| Participation                       | More direct        | Through voting          |
| Validator set                       | Can be larger      | Usually smaller         |
| Finality speed                      | Often fast         | Usually very fast       |
| Centralization risk                 | Medium             | Higher if few delegates |

In PoS, users become validators by staking.

In DPoS, users usually vote for delegates who validate on their behalf.

How such a leader is chosen depends on the respective blockchain system or pseudo-random factor

Validators/ Delegates

validators may distribute part of their reward to users who voted for them.

The number of times a particular BP (block producer) is selected to produce a block is proportional to the total votes received from the token holders

create a block in pseudo-random fashion within that particular round

## 54. EOS Block Validation

After one BP produces a block, other BPs validate it.

They check whether the block is valid.

A block is confirmed only when more than two-thirds of BPs agree.

The lecture writes this as:

(+2/3) majority

This means:

More than two-thirds majority.

## 55. What Does +2/3 Majority Mean?

EOS has 21 BPs.

Two-thirds of 21 is:

$$21 \times 2/3 = 14$$



More than two-thirds means at least 15 BPs.

So approximately:

15 out of 21 BPs must agree.

Once they agree, the block is confirmed/final.

## 57. Bitcoin vs EOS Finality

Page 32 compares Bitcoin and EOS.

| Blockchain | Consensus | Avg. Time Per Block | Avg. Time to Finality        |
|------------|-----------|---------------------|------------------------------|
| Bitcoin    | PoW       | 10 minutes          | 60 minutes, 6 confirmations  |
| EOS        | DPoS      | 0.5–1 second        | 2–3 seconds, 2–3 commitments |

This is one of the most important comparison tables in the lecture.

EOS has faster finality because: It has only 21 block producers

## 59. PoW vs PoS vs DPoS Summary

| Feature       | PoW                         | PoS                       | DPoS                              |
|---------------|-----------------------------|---------------------------|-----------------------------------|
| Main resource | Computing power/electricity | Staked coins              | Votes/delegates                   |
| Block creator | Miner                       | Validator/staker          | Delegate/block producer           |
| Energy use    | High                        | Lower                     | Lower                             |
| Penalty       | Weak direct penalty         | Stake can be lost         | Delegate can be penalized/removed |
| Finality      | Slower/probabilistic        | Faster                    | Very fast                         |
| Example       | Bitcoin                     | Ethereum after transition | EOS                               |

# Lecture-12

## Bitcoin

Input state: list of coins available for spending (UTXO set)

Transaction: set of instructions for updating the UTXO set

Output state: Transfer of Bitcoin to one or more accounts updating the UTXO set

Yes, Bitcoin script is a 'contract' in the sense that it provides programmable conditions for redeeming a coin but conditions are limited

- Can we use Bitcoin to
  - pay out a coin iff (if and only if) a user has a signing key? ☒
  - implement a second currency/asset? ☒
  - pay out a coin iff a candidate wins the US election? ☐
  - pay out a coin iff a majority of users vote to invest in a service? ☐

**Signing key:** Bitcoin natively uses digital signatures to authorize the spending of coins.

**Second asset:** Users can embed metadata via `OP_RETURN` to track external tokens on the ledger.

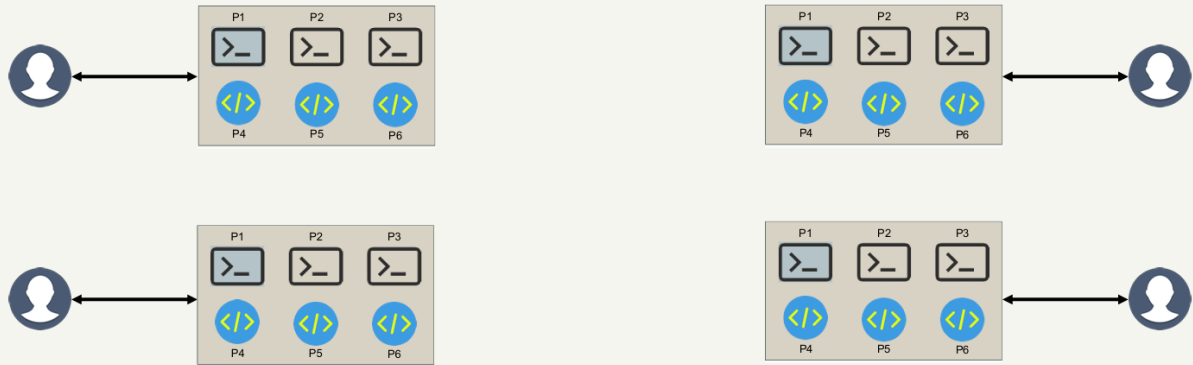
**Election result:** Bitcoin cannot access external data sources (the "Oracle problem") to verify real-world outcomes.

**Majority vote:** The protocol lacks a global state and complex logic to count votes across a dynamic user base.

What Ethereum is trying to do is to develop a world computer

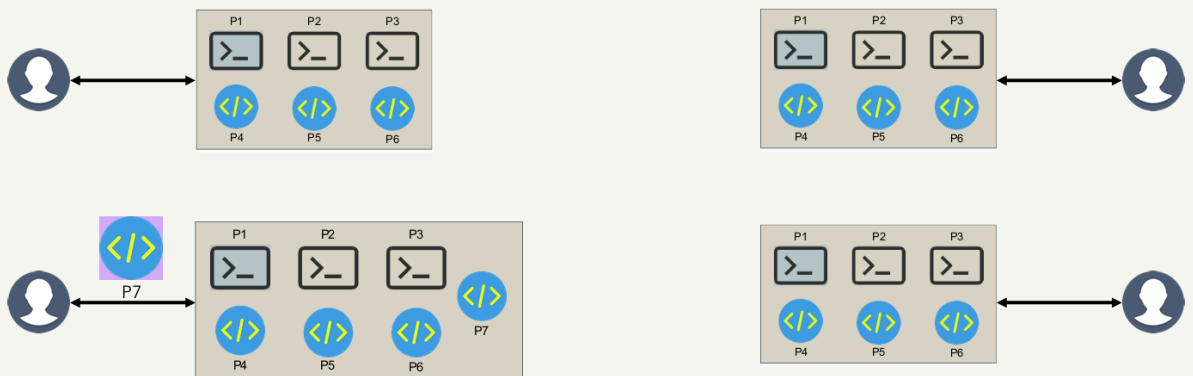
Decentralising the World Computer

## World computer



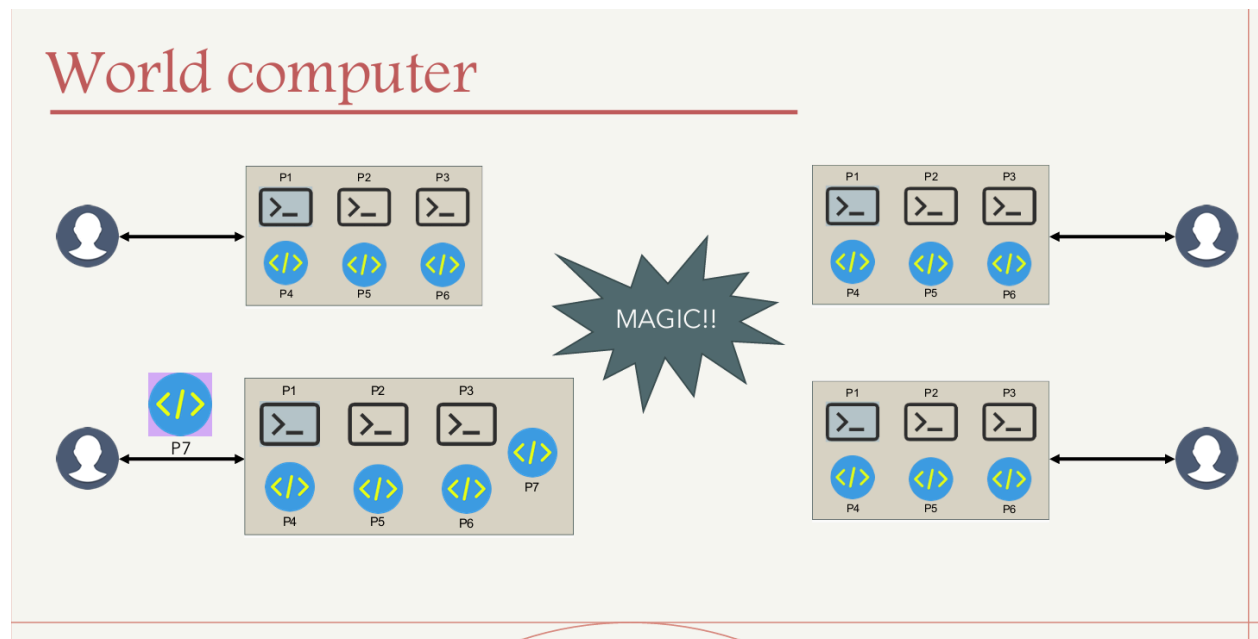
Instead of one big box, we now see four identical boxes. Each box is a **Node**. Every node runs the exact same set of programs (P1–P6).

## World computer



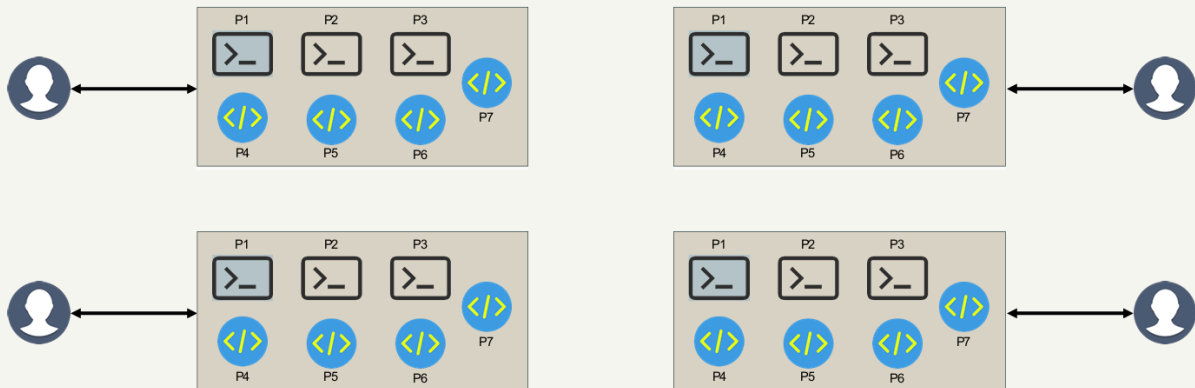
A user interacts with one specific node to deploy a new program, **P7**. The **Conflict**: Initially,  $P7$  only exists on the bottom-left node. The other three nodes

are now "out of sync"—they don't know \$P7\$ exists yet. In a decentralized system, everyone must agree on what programs exist and what their "state" (balance, data, etc.) is.



This "Magic" is the protocol that allows all nodes to communicate and agree on the new update (P7).

# World computer



After the consensus process, all four nodes now contain program **P7**. The system has reached **Consistency**

Ethereum consists of EVM and its blockchain. Ethereum was designed to act as a decentralized **"world computer"**. An Ethereum program is like any computer program.

To use the network, deploy code, or execute smart-contracts, users must pay transaction fees using Ethereum's native cryptocurrency, **Ether (ETH)**.

Data can be retrieved from the program. When a user sends a transaction containing data or commands to a smart contract, the contract executes its code based on its logic, which subsequently changes the "state" of the EVM. A consensus protocol is then used to ensure this new state and data are universally synchronized across every node in the blockchain.

Blockchain-based EVM distribution allows Ethereum to act as a decentralised World Computer!

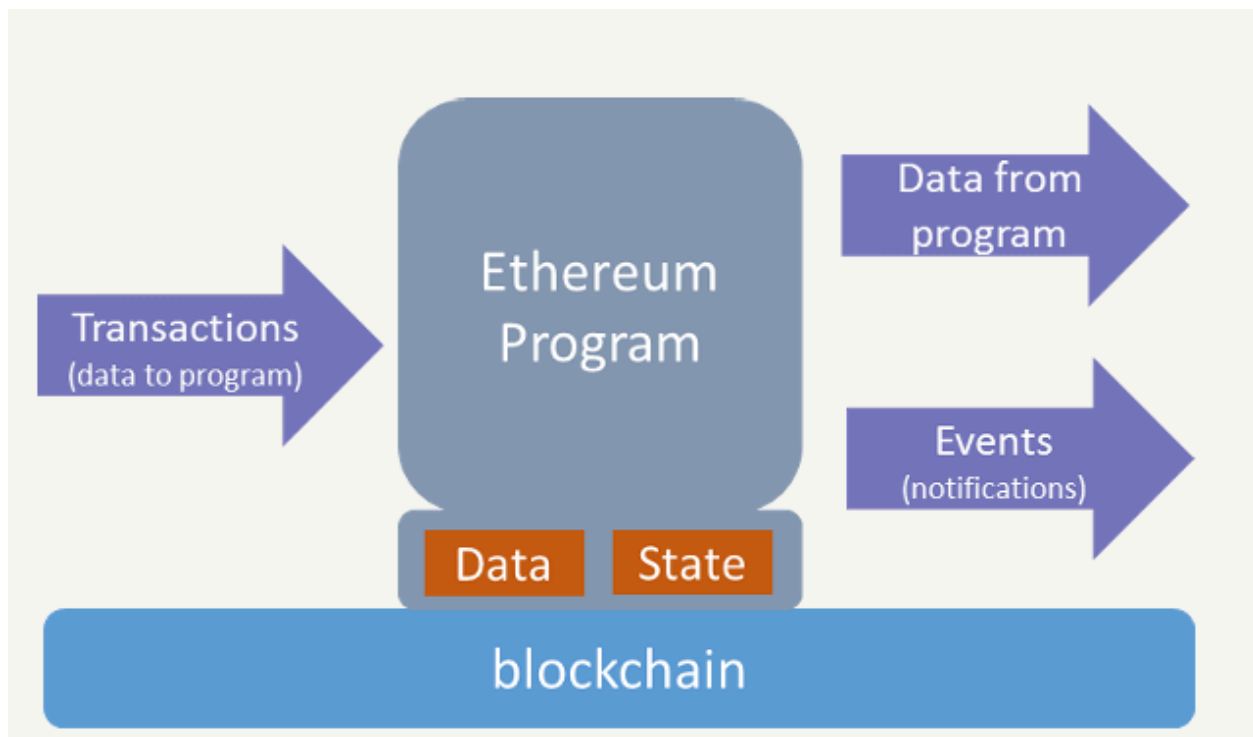
Being based on blockchain means once a code is deployed in the EVM, it is replicated to all nodes running the EVM.

This means Ethereum is **Turing-complete**. While a traditional computer runs code on a local CPU (like your laptop), Ethereum runs code on a **Virtual Machine (the EVM)** that exists globally.

2. "Can be invoked with data to perform specified operations on the supplied data which can be retrieved"

This describes the **execution and output** of a smart contract:

- **Invoked with data:** To run a program, you send a transaction to the contract's address containing "input data." For example, if the program is a calculator, your input data might be "add 5 and 10."
- **Perform specified operations:** The EVM executes the logic defined in the contract. In our example, it performs the addition.
- **Data which can be retrieved:** The result of that operation can either be stored permanently on the blockchain (a **state change**) or sent back to the user as a return value.



## 39. Storing Data in the World Computer

Ethereum lets smart contracts store data on-chain.

Example:

A token contract stores:

```
balances[Alice] = 100  
balances[Bob] = 50
```



This data is stored on the Ethereum blockchain and synchronized across nodes.

## 45. Replication Across Nodes

In Ethereum, deployed smart contract code is replicated to nodes running the EVM.

This means many nodes have the same contract code and state.

So even if one node fails, the system can continue.

Simple idea:

```
Contract deployed once  
↓  
Network consensus spreads it  
↓  
Nodes store/execute the same contract
```



This gives Ethereum strong availability.

## 48. Deploying Code

Deploying code means:

Putting smart contract code onto the blockchain.

In normal software:

Install app on computer/server



In Ethereum:

Deploy smart contract to blockchain



Deployment itself is done through a transaction.

It costs ETH because it uses computation and storage.

In Ethereum, transactions are used to interact with smart contracts.

A transaction can:

1. Send ETH
2. Deploy a smart contract
3. Call a smart contract function

So transactions act as commands sent to Ethereum programs.

In Ethereum, events are logs emitted by smart contracts.

When a transfer happens, the contract can emit an event.

DApps can listen for these events.

Events help external apps know what happened on-chain.

So smart contracts can both store and provide data.

Once code is deployed on the blockchain, it is hard or impossible to change directly.

So even though each node runs locally, all nodes must agree globally.

## Calling a Function

Smart contracts contain functions.

A user can call this function by sending a transaction.

The contract executes the function and updates state.

Deploying smart contracts costs ETH.

A smart contract can be invoked with data.

smart contracts are irreversible once part of the blockchain.

### 66. Ethereum Components

The lecture lists Ethereum components:

| Component                     | Meaning                          |
|-------------------------------|----------------------------------|
| Users and code representation | Accounts and smart contracts     |
| EVM                           | Executes smart contract bytecode |
| Blockchain                    | Stores transactions and state    |
| Node/Network                  | Computers maintaining Ethereum   |
| Consensus                     | Agreement mechanism among nodes  |

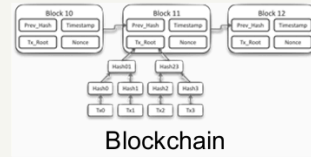
# Ethereum components



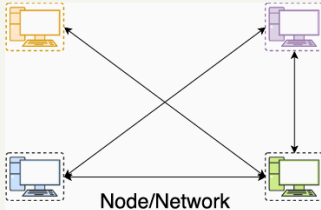
Users & code representation



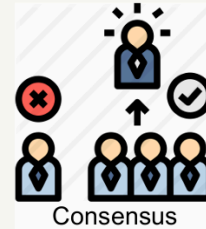
EVM



Blockchain



Node/Network



Consensus

## Lecture -13

Solidity code is written by developers → compiled into EVM bytecode → executed by the EVM.

Ethereum stores transaction history and account states in a blockchain.

## 1. Users and Code Representation

Ethereum has users, but it also has **programs** called **smart contracts**.

So in Ethereum, an address can represent:

| Type                   | Meaning                                       |  |
|------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------|
| User account           | Controlled by a human using a private key     |                                                                                     |
| Smart contract account | Controlled by code deployed on the blockchain |                                                                                     |

This is different from Bitcoin, where the main purpose is sending and receiving coins. Ethereum can also run programs.

## 2. EVM

EVM means **Ethereum Virtual Machine**.

It is the environment where smart contract code runs.

Think of it like this:

Solidity code is written by developers → compile ↓ → EVM bytecode → executed by the EVM.

Ethereum nodes must agree on:

Which transactions are valid and what the current state of the blockchain is.

Ethereum protocol follow kore holo Geth and Open Ethereum

Virtual machine is a software-based computer.

Deterministic —> Same node same input —> Same output

Solidity code → Ethereum compiler → EVM bytecode → EVM execution

Logs are used to emit events.

An EOA can sign transactions.

Answer: True

Contract accounts can receive and send Ether.

Answer: True

Smart contracts have persistent internal storage.

Answer: True

EVM memory is: Temporary during execution

Contract storage is: Permanent

Every transaction in Ethereum starts from an EOA.

A smart contract cannot start working by itself. It needs to be triggered by an EOA or another contract.

Why nonce is important:

It prevents replay attacks.

For EOA accounts, storage root is usually empty because EOAs do not store contract data.

## 9. Ethereum Network

Ethereum is a peer-to-peer network.

That means many computers connect with each other directly.

These computers are called **nodes**.

The slide shows different types:

| Node Type            | Meaning                                            |
|----------------------|----------------------------------------------------|
| Full node            | Stores and verifies the blockchain                 |
| Light node           | Does not store full blockchain                     |
| Mining pool provider | Coordinates miners in older Proof-of-Work Ethereum |
| Solo miner           | Mines independently in older Ethereum              |
| Pool participant     | Miner participating in a mining pool               |

The lecture uses older mining terminology. In current Ethereum, mining has been replaced by validator-based Proof of Stake, but the network-node idea is still important.

To connect to Ethereum, a computer must run Ethereum client software.

The lecture mentions two implementations:

Geth & OpenEthereum

For most private people, light nodes are the most comfortable way of interacting with the Ethereum blockchain

To connect to Ethereum, a computer must run Ethereum client software.

An implementation is basically software that follows Ethereum rules.

Ethereum is not one single software. It is a protocol, and different software clients can implement it.

Not all Ethereum nodes are using the same code base

Virtual machine —> Software Based Computer

Turing complete means the system can run general logic and solve many types of computational problems.

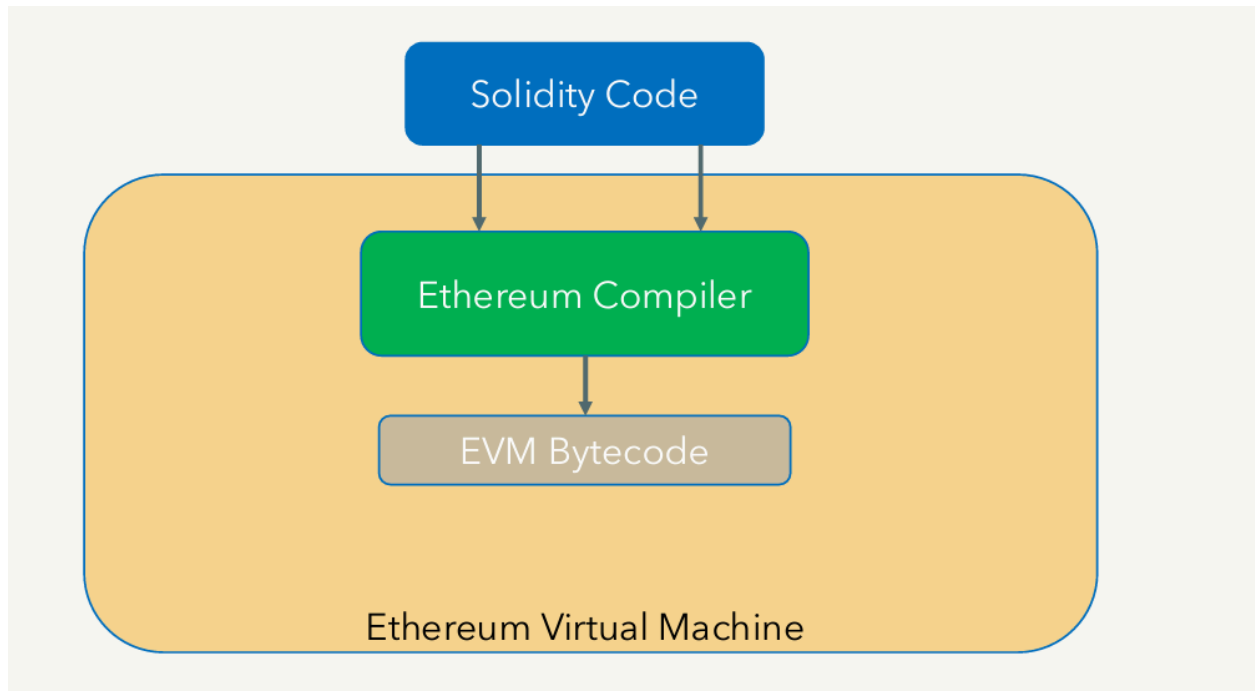
In simple words:

You can write complex programs using Ethereum smart contracts.

But because programs can contain loops, there is a risk:

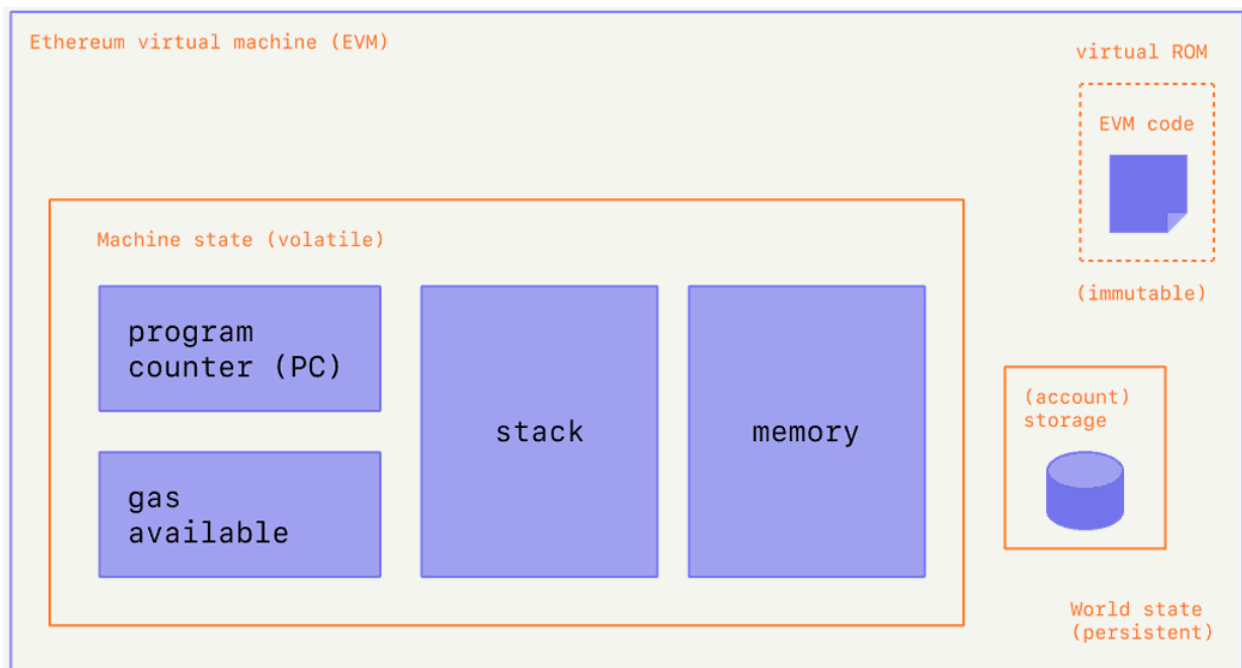
A program may run forever.

That is why Ethereum uses **gas**.



The EVM works mainly with 256-bit words.

This means its basic data size is 256 bits.



## 16. EVM Internal Structure

The EVM diagram shows several components:

| Component       | Meaning                                 |
|-----------------|-----------------------------------------|
| Program Counter | Tracks current instruction              |
| Stack           | Temporary working area                  |
| Memory          | Temporary data storage during execution |
| Storage         | Permanent contract storage              |
| Gas available   | Remaining gas for execution             |
| EVM code        | Contract bytecode                       |
| World state     | Persistent blockchain state             |

EVM is a LIFO machine

Memory is temporary.

It exists only while the contract function is executing.

After execution ends, memory is cleared.

Storage is permanent.

It remains on the blockchain after execution.

Gas available means how much execution fuel is left.

EVM code means the bytecode of the smart contract.

Once deployed, the contract code is usually immutable, meaning it cannot be changed directly.

Logs are used to emit events.

## 13. Node Discovery

When a new Ethereum node joins the network, it needs to find other nodes.

This is called **node discovery** or **peer discovery**.

Problem:

| If a node knows nobody, it cannot join the network.

Solution:

Ethereum clients have some default peers hardcoded.

The slide says Geth has hardcoded peers.

Basic process:

1. New node looks up hardcoded nodes
2. It sends a hello message
3. The other node replies with acknowledgement
4. The new node becomes connected to the network

So node discovery is like entering a new classroom and asking someone:

| "Can you introduce me to others?"

The EVM can access some blockchain information.

Examples:

| Information  | Meaning                  |
|--------------|--------------------------|
| address      | Current contract address |
| balance      | Balance of account       |
| block number | Current block number     |
| gas price    | Transaction gas price    |

External Transaction can :

| Type                | Meaning                       |
|---------------------|-------------------------------|
| Regular transaction | Sending ETH                   |
| Contract deployment | Creating a new smart contract |
| Contract execution  | Calling a deployed contract   |

### External Transaction Structure

External transaction a main role play korbe originator i

## D. to

The `to` field is the recipient address.

For a regular ETH transfer:

| `to` = receiver's account address.

For contract execution:

| `to` = smart contract address.

For contract deployment:

| `to` field is empty.

The `value` field means the amount of Wei transferred.

The `data` field is optional.

It is used for:

| Transaction Type     | Use of Data Field           |
|----------------------|-----------------------------|
| Contract deployment  | Contains contract bytecode  |
| Contract execution   | Contains function call data |
| Regular ETH transfer | Usually empty               |

## A. Regular Transaction

This means sending Ether from one account to another.

Example:

```
Alice sends 1 ETH to Bob.
```



The `to` field contains Bob's address.

## B. Contract Deployment Transaction

This creates a new smart contract.

Special feature:

It has no `to` address.

Why?

Because the contract does not exist yet.

The contract code is placed in the `data` field.

After deployment, Ethereum creates a new contract address.

## C. Contract Execution Transaction

This calls a function of an existing smart contract.

Example:

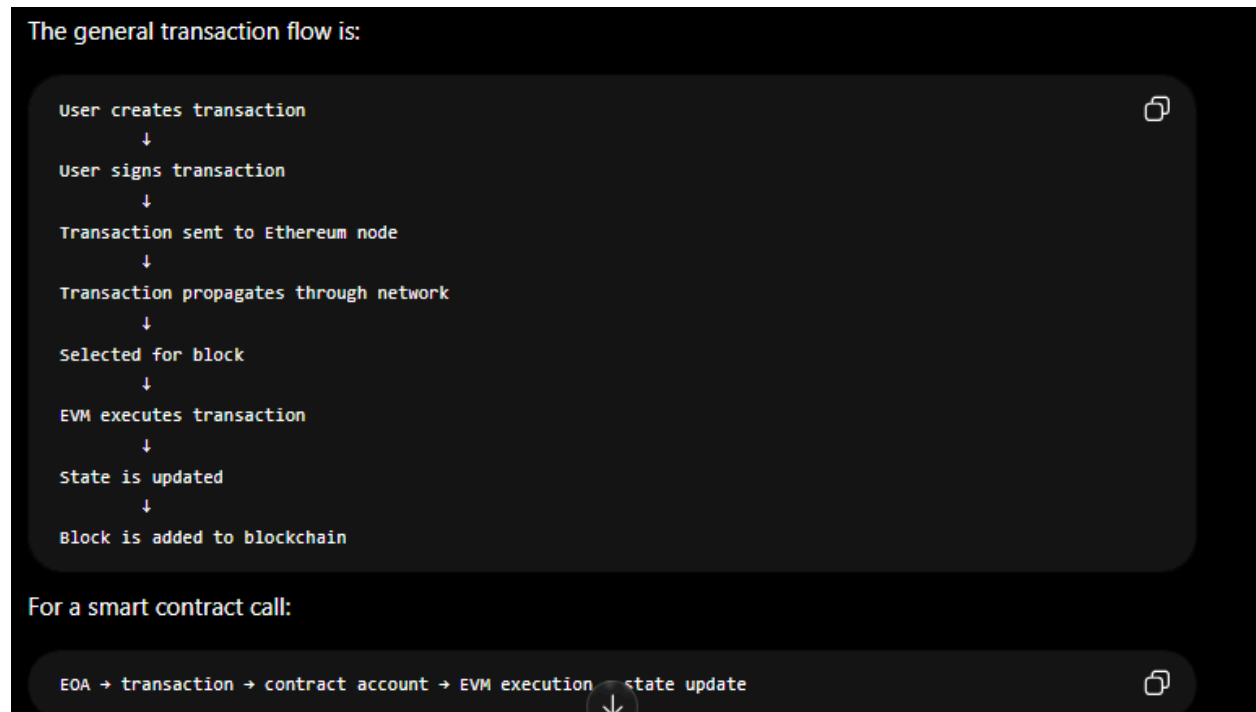
```
</> solidity  
vote(candidateId)
```



In this case:

| Field              | Meaning                     |
|--------------------|-----------------------------|
| <code>to</code>    | Smart contract address      |
| <code>data</code>  | Function name and arguments |
| <code>value</code> | ETH sent, if any            |

Internal transactions are not serialized and directly put on the blockchain like external transactions. But if they change account balances or storage, the final state change is stored on the blockchain.



In order to ensure safety, a type of DoS protection, the concept of fee/gas is used

**Pay as you execute** : Meaning users need to pay for both computation and storage of data .This payment is collected by the respective miner

One hash call requires 30 gas . Ethereum also uses gas for contract creation

**Gas measures work.**

**Ether pays for gas.**

The gasLimit is the maximum gas the sender allows the transaction to use.

Fee for computation in Ether = gasLimit x gasPrice which is placed in escrow

|                                   |                                       |
|-----------------------------------|---------------------------------------|
| Execution succeeds                | Used gas is paid, unused gas refunded |
| Execution fails due to out-of-gas | State reverts, gas fee is consumed    |

### 33. What Happens If Gas Runs Out?

If gas runs out before the transaction finishes:

1. Execution stops
2. State changes are reverted
3. Gas fee is not refunded

Example:

Suppose a contract tries to update a value.

If gas runs out before completion:

The value goes back to its previous state.

But the user still pays for the consumed gas.

Why?

Because Ethereum nodes still spent computational effort trying to execute it.

### 34. What Happens If Execution Completes?

If execution completes successfully:

1. The transaction changes the state
2. Miner/block producer receives fee for consumed gas
3. Remaining gas is refunded to sender

Example:

```
gasLimit = 100,000  
gas used = 60,000  
gasPrice = 10 Gwei
```



Actual fee:

```
60,000 × 10 Gwei = 600,000 Gwei
```



Refund:

```
40,000 × 10 Gwei = 400,000 Gwei
```



Before execution:

$\text{gasLimit} \times \text{gasPrice}$

is reserved from the sender.

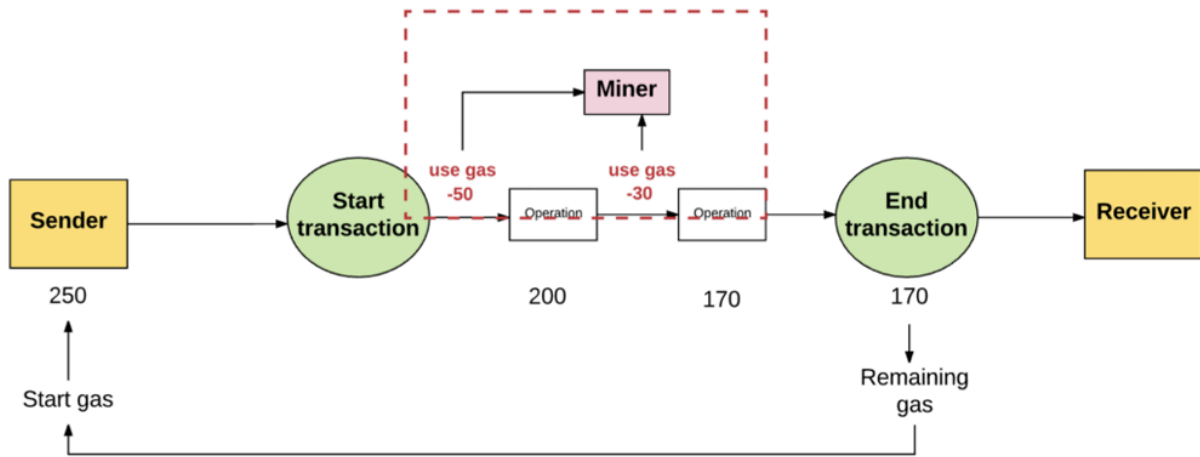
- **Gas limit:** Max no. of computational steps the transaction is allowed.
- **Gas Price:** Max fee the sender is willing to pay per computation step.

|                            |          |                             |          |                                           |
|----------------------------|----------|-----------------------------|----------|-------------------------------------------|
| Gas Limit<br><b>50,000</b> | <b>X</b> | Gas Price<br><b>20 gwei</b> | <b>=</b> | Max transaction fee<br><b>0.001 Ether</b> |
|----------------------------|----------|-----------------------------|----------|-------------------------------------------|

| Concept   | Meaning                   |
|-----------|---------------------------|
| Gas       | Unit of work              |
| Gas price | Ether paid per gas        |
| Ether     | Currency used for payment |
| Wei       | Smallest unit of Ether    |

Out of Gas ----> Changes are reverted ( no Refund )

All the money spent on gas by the sender is sent to the miner's address.





# Lecture -14

## Block

### Header + Body

#### Header

Parent Hash :

Purpose of Gas :

Prevents infinite loops  
Prevents spam attacks

Nonce : A special number used in p0w that the miner changes repeatedly during mining

### H. MixHash

Definition:

A hash proving sufficient computation was performed.  CSE446 Lecture 14 (1)

Works together with nonce.



Bloom Filter ( A probabilistic data structure for fast searching ) storing log information

## Gas Limit

Maximum gas allowed inside one block.

Purpose:

- Prevents overly large blocks
- Controls computation load

## A. State Root

Definition:

Hash of the root node of the state trie.

Contains:

- account balances
- nonces
- contract data

## C. Receipt Root

Definition:

Hash of transaction receipt trie.

Contains:

- gas used
- execution results
- logs

## B. Transaction Root

Definition:

Hash of the transaction trie root.

Contains all block transactions.

### Why Use Root Hashes?

Because changing even one transaction changes the root hash.

### Ethereum Ommers (Uncle Blocks)

#### Bitcoin

Losing block:

- discarded completely
- miner gets no reward

---

#### Ethereum

Ethereum partially rewards the ommer miner.

Why?

Because miner still contributed computational work.

## Why Ethereum Has More Ommers

Ethereum block time:

~15 seconds



Bitcoin block time:

~10 minutes



 CSE446 Lecture 14 (1)

Shorter block times mean:

- faster transactions
- but more simultaneous blocks
- therefore more forks

Hence Ethereum introduced ommers.

An ommer is a block whose parent is equal to the current block's parent's parent .Current Block er grandparent is equal to the ommers parent

## 6. Trie

CSE446 Lecture 14 (1)

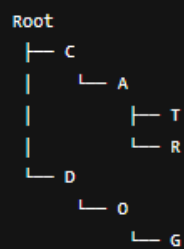
A Trie is a tree-like data structure storing keys character by character.

### Example

Words:

CAT  
CAR  
DOG

Trie:



A Patricia (Practical Algorithm To Retrieve Information Coded In Alphanumeric ) trie is a binary radix trie

Binary Radix —> binary choice at each node

**root node becomes a cryptographic fingerprint of the entire data structure**

## Benefits

 CSE446 Lecture 14 (1)

- fast prefix searching
- small memory usage
- easy implementation

Used in:

- routers
- routing tables
- blockchain systems

## MPT Properties

 CSE446 Lecture 14 (1)

- stores key-value pairs
- cryptographically authenticated
- tamper resistant

### Why Trie is Useful

Common prefixes are shared.

Nodes in MPT can have 16 child nodes + its own value = 17

## Why 16 Children?

Ethereum uses hexadecimal.

Hex digits:

0-9, a-f

Total:

16 symbols

A hex character is called a nibble. Hex Expression a prottek ta character ke nibble bola hoy

Leaf node stores final value . Uses the hash function KECCAK256 ()

Node structures : -

## 11. Branch Node Structure

CSE446 Lecture 14 (1)

Branch node contains:

| Field | Purpose               |
|-------|-----------------------|
| 0-f   | possible child paths  |
| value | optional stored value |

Each field corresponds to one hexadecimal digit.

## 12. Leaf Node Structure

CSE446 Lecture 14 (1)

Leaf node contains:

- prefix
- key-end
- actual value

Example:

| prefix | key-end | value  |
|--------|---------|--------|
| 2      | ef      | 20 ETH |

Ethereum uses four tries.

Ethereum branch nodes have:

16 child slots + 1 value field

MPT

```
Branch
↓
Remove used nibble
↓
Only one path?
├─ YES → Leaf
└─ NO
    ↓
  Shared prefix?
  └─ YES → Extension
  └─ NO
      ↓
    Branch again
```

### **A. State Trie**

Stores:

- account balances
- nonces
- contract references

---

### **B. Transaction Trie**

Stores all transactions in block.

---

### **C. Receipt Trie**

Stores transaction receipts.

---

### **D. Account Storage Trie**

Stores smart contract storage variables.

## **14. Bitcoin Blockchain vs Ethereum Blockchain**

Bitcoin mainly stores:

- transactions

Ethereum stores:

- transactions
- smart contracts
- world state
- tries
- receipts

So Ethereum blockchain structure is more complex.

pow/Compute-bound consensus algorithm

## ASIC

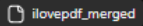
Special hardware optimized only for mining.

ASIC miners are much faster than normal computers.

rich getting richer” scenario

# Lecture - 15

## 1. Memory-Bound Consensus Algorithm

Bitcoin originally used a CPU-bound Proof of Work system. Ethereum tried to improve this using a **memory-bound PoW**. 

### What is CPU-bound?

A CPU-bound algorithm mainly depends on processor power.

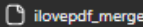
Example:

- Faster CPU = faster mining
- ASIC miners become extremely powerful
- Mining becomes centralized

### What is Memory-bound?

A memory-bound algorithm depends more on:

- RAM size
- Memory bandwidth
- Memory latency

instead of only raw processor speed. 

### Main idea

Even if someone has many ASIC chips, memory becomes the bottleneck.

So:

- Huge ASIC advantage decreases
- Ordinary GPU miners can compete better
- Mining becomes more decentralized

## 2. Ethereum PoW Consensus: Ethash

Ethereum originally used a PoW algorithm called **Ethash**.

ilovepdf\_merged

It combined:

- Dagger
- Hashimoto

Hence the name:

**Dagger-Hashimoto (Ethash)**

---

### DAG (Directed Acyclic Graph)

Ethereum mining used a huge data structure called a DAG.

A DAG is:

A directed graph with no cycles.

ilovepdf\_merged

Simple meaning:

- Data flows only forward
- You cannot loop back

## 3. How Ethash Works

### Step 1: Create DAG

Ethash creates a large pseudo-random dataset called the DAG.

#### Epoch

An epoch = 30,000 blocks  
≈ about 5 days.

After every epoch:

- New DAG is generated
- 

## Step 2: Generate Seed

Ethereum generates a seed based on chain length.

That seed helps generate:

- 16 MB cache
- 

## Step 3: Build DAG

Using the cache:

- DAG entries are generated pseudo-randomly.
- 

## Step 4: Mining Process

Ethereum miner takes:

- Block header
- Nonce

and hashes them using:

- Keccak (SHA-3) hash function
- 

## Step 5: Mix with DAG

The hash is repeatedly mixed with DAG data.

This creates:

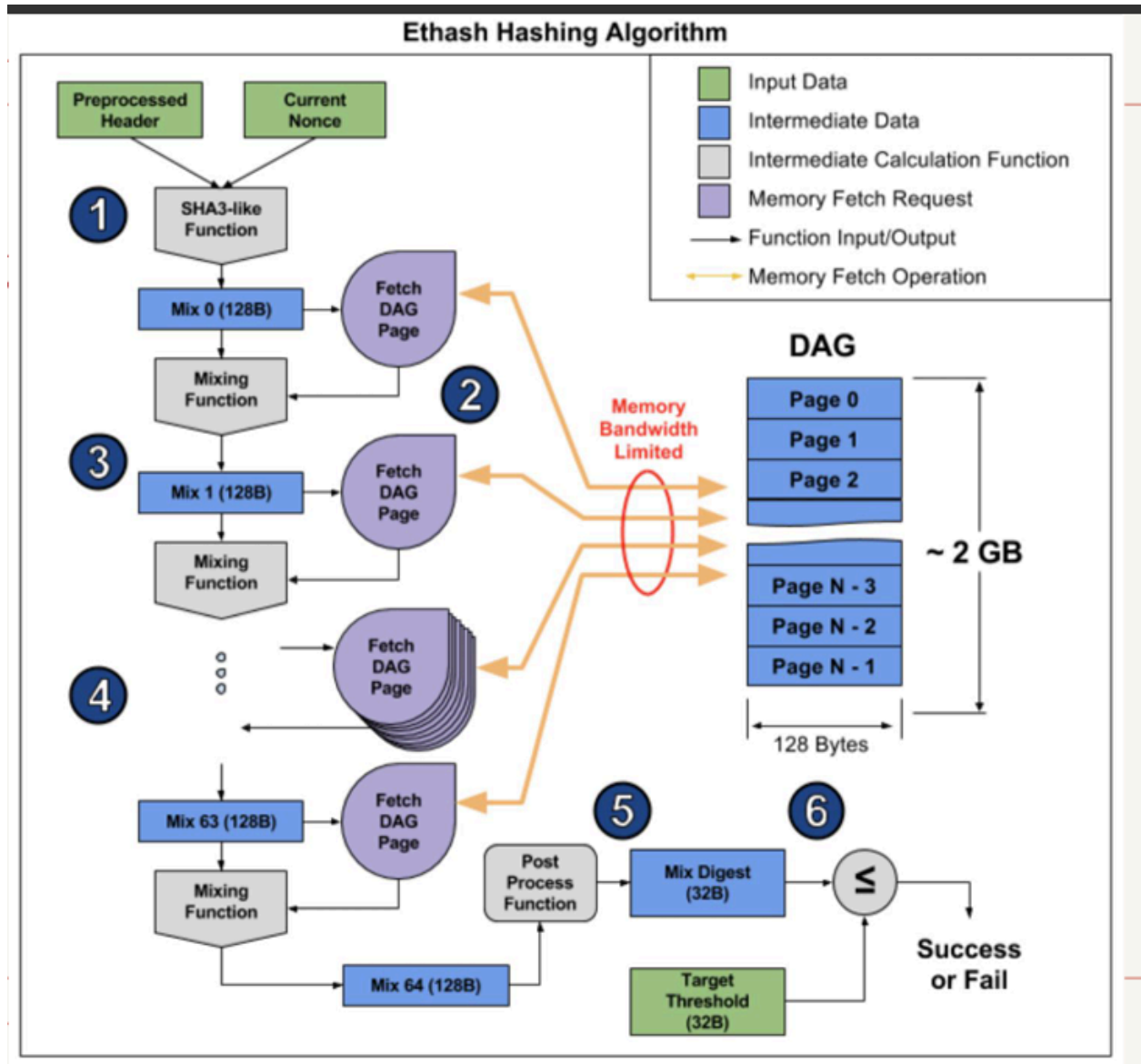
- mixHash
-

## Step 6: Compare with Difficulty

Final hash is checked against difficulty target.

If:

- $\text{hash} < \text{target} \rightarrow \text{valid block}$
- otherwise rejected



## Important Ethereum Terms

| Term           | Meaning                           |
|----------------|-----------------------------------|
| Ethash         | Ethereum's old PoW algorithm      |
| DAG            | Large mining dataset              |
| Nonce          | Random number miners change       |
| Validator      | Person staking ETH in PoS         |
| Slot           | 12-second unit in PoS             |
| Epoch          | 32 slots                          |
| Attestation    | Validator vote                    |
| Slashing       | Penalty for bad validators        |
| World Computer | Shared decentralized computer     |
| State          | Current data/status of blockchain |

## Lecture -16

Disrupting the existing method brings innovation which leads to new service delivery models

## 14. Why Private Blockchain?

Public blockchains like Bitcoin/Ethereum are not ideal for enterprises.

 ilovepdf\_merged

Companies need:

- privacy
- known participants
- high speed
- low latency
- permissioned access

## 15. Enterprise Requirements

Important requirements:

 ilovepdf\_merged

### Identifiable Participants

Users must be known.

Example:

- Banks require KYC/AML compliance.

### Permissioned Network

Only authorized members can join.

### High Throughput

Many transactions per second.

### Privacy

Business transactions must stay confidential.



Know-Your-Customer (KYC) and Anti-Money Laundering (AML)

# Enterprise-grade DLT

Let's break it into parts.

## DLT

DLT stands for:

## Distributed Ledger Technology

Meaning:

- A ledger/database is shared across many computers (nodes)
- Every participant keeps a copy
- Data is synchronized among them
- No single central authority controls it

Blockchain is one type of DLT.

## Enterprise-grade

Fabric is modular because its components are independent and can be customized or replaced without changing the entire blockchain system.

Hyperledger Fabric allows you to customize the specific rules for how transactions are approved and verified for each individual application without affecting the rest of the network

It also enables confidentiality through its channel architecture

A specific subset of participants can create a channel to conduct business, and **only the nodes involved in that specific channel are granted visibility into its smart contracts (chaincode) and transacted data**

**values within the chaincode can be partially or completely encrypted**

Once the encrypted data is permanently written to the blockchain ledger, **it can only be read and decrypted by users who possess the specific corresponding key**

Fabric relies on a concrete identity management architecture

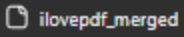
## 22. Identity Management

Fabric uses:

**PKI (Public Key Infrastructure)**

and:

**Certificate Authorities (CA)**

to verify identities. 

Think of this as the "ID Card Office." Once the MSP proves *who* you are, the ACL determines *what* you are allowed to do. It maps specific identities or roles to specific actions.

**Granular Control:** Instead of just having "all-access" or "no-access," administrators can fine-tune permissions for every specific task within the network.

there must be a way to guarantee that a certain public key represents an authorised entity in a network

## Digital Certificate

A digital certificate is like:

**a digital ID card**

It says:

"This public key belongs to Rahim from Organization R1"



The lecture gives an example like:

Anybody can utilise the public key of the CA to verify the signature in the certificate  
If the certificate is tampered, the signature verification will fail

We used the signed certificate to get the public key of an entity

Circular Problem —> Solution : hierarchical

Committing peers do NOT decide transaction order.

- An endorsing peer is always also a committing peer

## A. Collect Transactions

Applications send endorsed transactions to orderer.

## B. Order Transactions

Orderer:

- sorts transactions
- serializes them

## C. Create Blocks

Transactions grouped into blocks.

## D. Broadcast Blocks

Blocks sent to peers in channel.

 ilovepdf\_merged

## Fabric Network

We start a network with the initialisation of an orderer

A consortium means:

A group of organizations working together for a shared blockchain purpose.

Only organizations with proper permissions can create consortiums. ( jader already permissions ase )

channel configuration CC1, is completely separate to the network configuration

A peer is: a node that stores ledger and runs chaincode

There can be many ledgers and chaincodes hosted on an individual peer

Each ledger can have many chaincodes which access it

In Hyperledger Fabric, a ledger consists of two distinct, though related, parts • a world state and a blockchain

A world state is a database that holds a cache of the current values of a set of ledger states

Ledger states are, by default, expressed as key-value pairs

### **Important Concept**

Each peer maintains:

**its own copy of ledger**

This creates:

**distributed ledger**

## Lecture -17

Ledger manei Database

## 2. Fabric Network Basics

A Fabric network contains:

| Component          | Purpose                                    |
|--------------------|--------------------------------------------|
| Peer               | Stores ledger and executes smart contracts |
| Ledger             | Stores blockchain data                     |
| Chaincode          | Smart contract                             |
| Channel            | Private communication layer                |
| Ordering Service   | Orders transactions into blocks            |
| MSP                | Identity management                        |
| Client Application | Sends transaction requests                 |

Fabric ledger has **two parts**:

1. **World State**
2. **Blockchain**

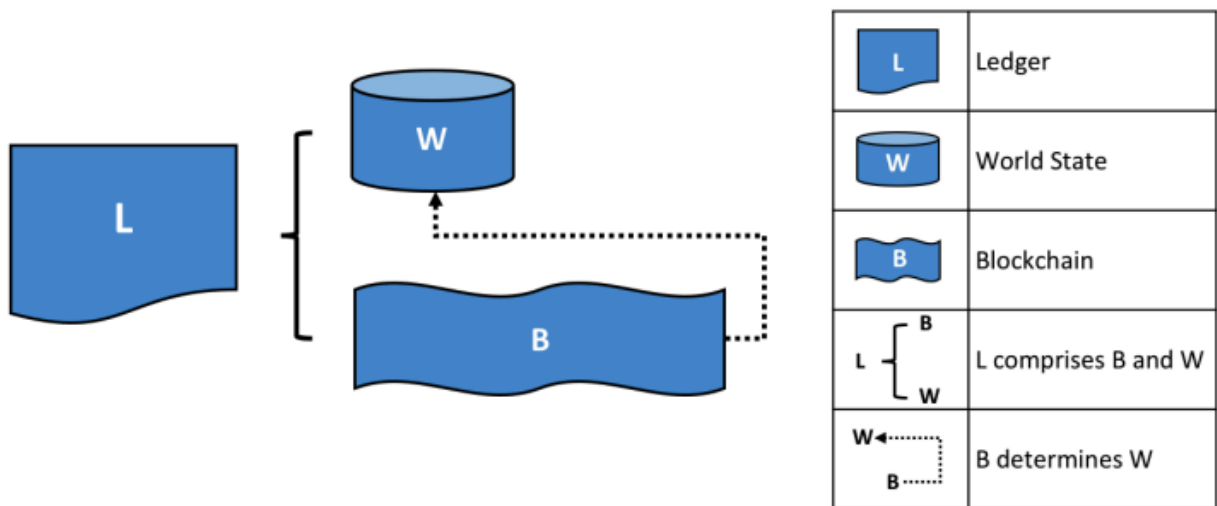
## 6. Initial Ledger State

Initially:

- blockchain exists
- world state is empty

After transactions:

- blockchain records history
- world state updates to latest values



## 9. Relationship Between Blockchain and World State

Slides say:

World State W is derived from Blockchain B



Meaning:

Blockchain = full history

World state = latest result

Example:

Blockchain history:

Current Block Hash er purpose hocche integrity Verification

Block Data is written when the block is created by the ordering service (block creation service)

## 18. Ordering Service

Ordering service creates blocks.

Responsibilities:

- collect transactions
- order them
- create blocks
- broadcast blocks

Even invalid transactions stay recorded.

If transaction changes:

- signature becomes invalid

Proposal = request sent to smart contract.

## 28. Why RW Set is Important

Fabric uses:

**Execute → Order → Validate**

instead of:

**Order → Execute**

RW set helps detect:

- conflicting transactions
- double spending
- stale data updates

## 31. Installing Smart Contract

Steps:

1. Develop chaincode
2. Install on peer
3. Instantiate on channel

CSE446 Lecture 17 (2)

## Lecture -18

Every node maintains a copy of the blockchain ledger.

### 1.1 Distributed Data Sharing

#### What it means

In blockchain, data is not stored in only one computer/server.

Instead, the same copy of data is stored across many computers called **nodes**.

Every node maintains a copy of the blockchain ledger.

#### How it works

Suppose:

- Alice sends 2 BTC to Bob
- The transaction is broadcast to the blockchain network
- Multiple nodes verify it
- After consensus, the transaction is added to a block
- That updated block is shared with all nodes

So every node gets the same updated data.

Data Persistence —> Blockchain data continues to exist as long as enough nodes remain alive.

## Example

Suppose:

- 10,000 Bitcoin nodes exist
- 2,000 go offline

Still:

- remaining 8,000 nodes preserve blockchain

## Difference between persistence & availability

### Persistence

Data survives long-term.

### Availability

Data can be accessed anytime.

## BlockChain Misconceptions

Truths :

Blockchain data can never change , transaction history & smart contract variables can change

Blockchain is NOT ideal for large storage.

Blockchain only guarantees: Stored data cannot be modified secretly afterward.

In a blockchain system, data encryption is not provided

## Wrong idea

“Blockchain automatically encrypts all data.”

## Reality

Blockchain mainly uses:

- hashing
- digital signatures

NOT encryption by default.

Performance of any database in terms of data access rate is much better than that of any blockchain system

It can guarantee the integrity of data only after it is stored in the blockchain

Digital signature is used for data provenance while a cryptographic hash is used to ensure data integrity

The power consumption of any private blockchain system will be comparable to any existing system

Private Blockchain usually normal server-level power usage.

## Blockchain limitations

### 3.1 Blockchain Bloating

#### Meaning

Blockchain size continuously increases over time.

Because blockchain is:

- Add-only database

Data is rarely deleted.

#### Example

Finding UTXO in the huge Bitcoin ledger becomes slower.

## 3.2 Blockchain Scaling Problem

### Meaning

Blockchain struggles to process very large numbers of transactions quickly.

Ethereum

Target Size → 45 million gas

The size of blocks will increase or decrease in accordance with network demands

Blockchain scalability trilemma was introduced by Vitalik

### SSD

#### Highly decentralized + secure

But:

- slower TPS

Example:

- Bitcoin

---

#### Highly scalable + secure

May reduce decentralization.

Example:

- some private chains

### Code Immutability

Once smart contract deployed: code cannot easily change

## **Advantage**

Prevents malicious modification.

## **DisAdvantage**

If bug exists bug becomes permanent

## **Solution**

The error needs to be fixed off-chain and then re-deployed in the network

This will result in a new contract address —> Update the Dapp

## **Usability**

strong usability issues with Blockchain wallets

- In a 2018 estimation:
  - When 1 ETH = 200 USD
  - 1 KB required 2 USD
  - 1 MB would cost around 2000 USD
- But now, 1 ETH = 2200 USD
  - It is very difficult to predict, how much it might cost
  - One option is to try via Ganache or in the test network

## **Blockchain Feasibility**

Should we actually use blockchain?"

# Lecture 19

malicious node cannot effect the transaction fully is its less than 50%

## 2. Creating a "Secret" Fork

Instead of broadcasting their new blocks to the public network, the attacker starts building their own private version of the blockchain.

- **The Public Chain:** Honest miners continue adding blocks here.
- **The Attacker's Chain:** The attacker works on a parallel chain in secret. Because they have more power, they can generate these blocks **faster** than the honest miners can grow the public chain.

Transaction Censoring Attack ———> Block Censoring Attack

The core development team of the blockchain system holds regular online meetings where anyone is welcome to participate